

Problem Set 3

Tate Mason

Invalid Date

Question 1

```
main_data <- read_csv(here("Y2S1/I0/PS3/Data/prod_level_data.csv"))
```

Rows: 71 Columns: 21

-- Column specification -----

Delimiter: ","

dbl (21): market, x1, x2, x3, x4, p1, p2, p3, p4, ave_dist1, ave_dist2, ave...

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
if (all(c("x", "p", "ave_dist", "s", "mc") %in% names(main_data))) {  
  main_long <- main_data  
} else {  
  main_long <- main_data %>%  
    pivot_longer(  
      cols = matches("^x|p|ave_dist|s|mc") %>%  
        names_to = c(".value", "prod"),  
        names_pattern = "(.*)\\d+$"  
    ) %>%  
    mutate(prod = as.integer(prod)) %>%  
    arrange(market, prod)  
}  
  
head(main_long)
```

```
# A tibble: 6 x 7
  market prod    x    p ave_dist    s    mc
  <dbl> <int> <dbl> <dbl>    <dbl> <dbl> <dbl>
1      1     1 2.02  8.71  0.0875 0.48  0.768
2      1     2 0.252 5.20  0.0865 0.098 0.824
3      1     3 3.47  5.10  0.0684 0.163 0.348
4      1     4 0.306 5.03  0.0474 0.12  0.464
5      2     1 0.942 5.43  0.553  0.143 0.721
6      2     2 0.909 4.80  0.966  0.039 0.672
```

```
describe(main_long)
```

	vars	n	mean	sd	median	trimmed	mad	min	max	range	skew
market	1	284	36.00	20.53	36.00	36.00	26.69	1.00	71.00	70.00	0.00
prod	2	284	2.50	1.12	2.50	2.50	1.48	1.00	4.00	3.00	0.00
x	3	284	2.57	1.48	2.55	2.58	2.00	0.03	4.99	4.96	-0.05
p	4	284	5.46	0.91	5.22	5.33	0.60	4.17	9.56	5.39	1.76
ave_dist	5	284	0.84	0.90	0.55	0.69	0.47	0.01	6.54	6.53	3.21
s	6	284	0.17	0.12	0.15	0.16	0.10	0.01	0.56	0.56	1.10
mc	7	284	0.52	0.29	0.55	0.52	0.35	0.00	1.00	0.99	-0.09

	kurtosis	se
market	-1.21	1.22
prod	-1.37	0.07
x	-1.28	0.09
p	3.87	0.05
ave_dist	14.91	0.05
s	0.73	0.01
mc	-1.20	0.02

```
stat_tab <- main_long %>%
  select(x, p, ave_dist, s, mc) %>%
  pivot_longer(
    everything(),
    names_to = "variable",
    values_to = "value"
  ) %>%
  group_by(variable) %>%
  summarize(
    N = n(),
    Mean = mean(value),
    SD = sd(value),
```

```

    Min = min(value),
    P25 = quantile(value, 0.25),
    Median = median(value),
    P75 = quantile(value, 0.75),
    Max = max(value),
    .groups = "drop"
  )
kable(stat_tab,
      digits = 3,
      caption = "Table of Summary Statistics")

```

Table 1: Table of Summary Statistics

variable	N	Mean	SD	Min	P25	Median	P75	Max
ave_dist	284	0.845	0.899	0.014	0.343	0.546	1.159	6.543
mc	284	0.516	0.289	0.004	0.273	0.549	0.765	0.996
p	284	5.460	0.906	4.168	4.861	5.224	5.755	9.557
s	284	0.174	0.120	0.007	0.086	0.148	0.224	0.564
x	284	2.568	1.478	0.030	1.222	2.548	3.934	4.993

```

main_graph_pxs <- main_long %>%
  ggplot(aes(x = x, y = s)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, color = "blue") +
  labs(
    title = "Price vs. Market Share by Product",
    x = "Market Share (s)",
    y = "Price (p)"
  ) +
  theme_minimal()

main_graph_xxs <- main_long %>%
  ggplot(aes(x = p, y = s)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, color = "blue") +
  labs(
    title = "Rating vs. Market Share by Product",
    x = "Rating (x)",
    y = "Market Share (s)"
  ) +

```

```

theme_minimal()

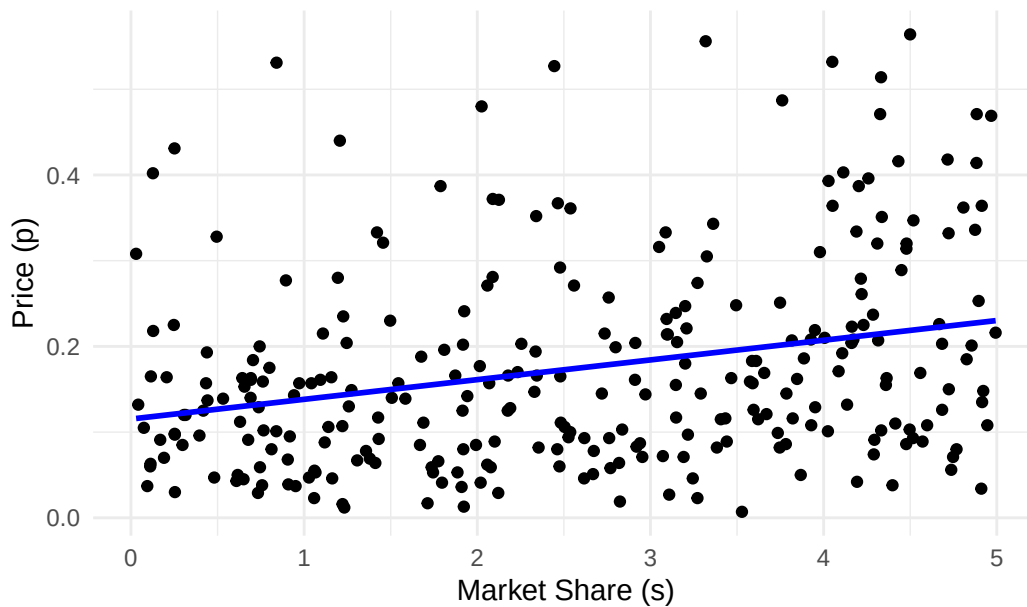
main_graph_sxad <- main_long %>%
  ggplot(aes(x = ave_dist, y = s)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, color = "blue") +
  labs(
    title = "Average Distance vs. Market Share by Product",
    x = "Average Distance (ave_dist)",
    y = "Market Share (s)"
  ) +
  theme_minimal()

main_graph_pxs

```

`geom\_smooth()` using formula = 'y ~ x'

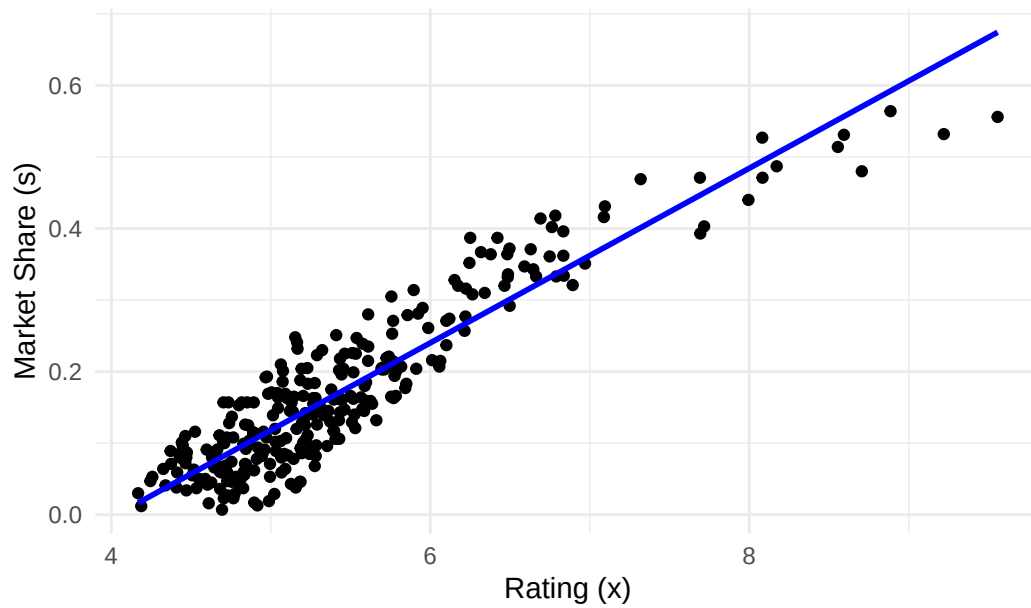
Price vs. Market Share by Product



```
main_graph_xxs
```

`geom\_smooth()` using formula = 'y ~ x'

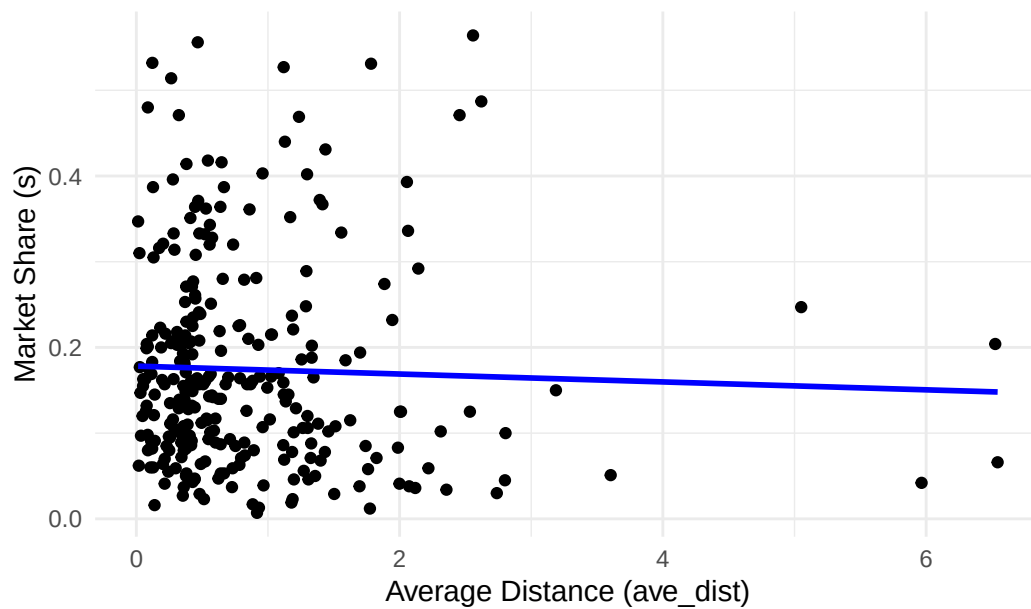
Rating vs. Market Share by Product



```
main_graph_sxad
```

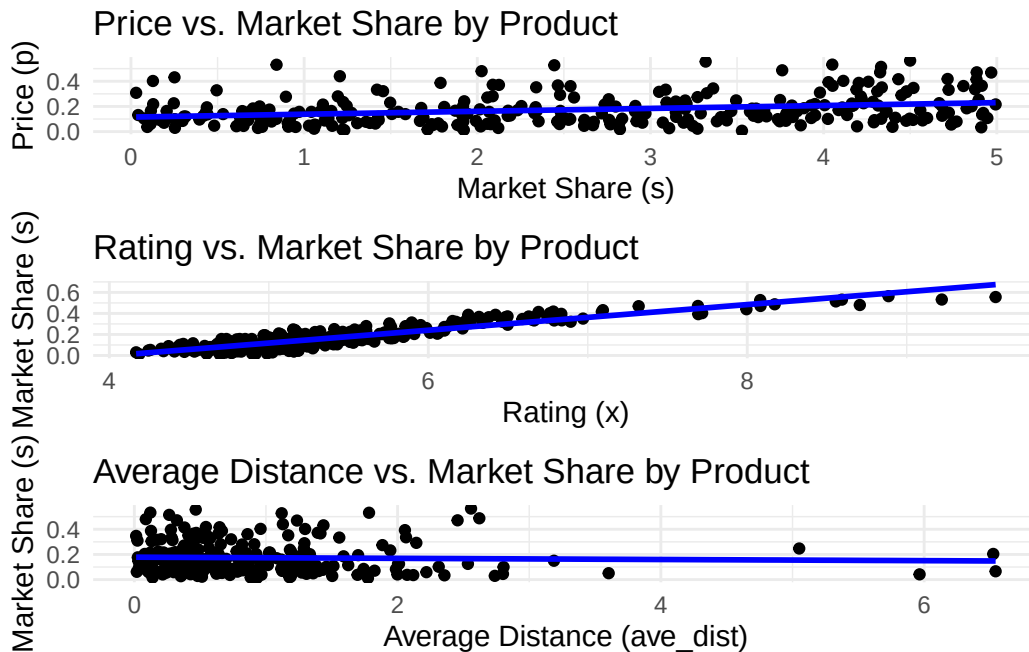
```
`geom_smooth()` using formula = 'y ~ x'
```

Average Distance vs. Market Share by Product



```
grid.arrange(main_graph_pxs, main_graph_xxs, main_graph_sxad, ncol = 1)
```

```
`geom_smooth()` using formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
`geom_smooth()` using formula = 'y ~ x'
```



Price is positively correlated with market share, with price increasing with share for all products. For Yelp rating, there is no discernable correlation. For distance, there is a slightly negative relationship, though it appears to be as good as flat. All make sense to me, as price is directly manipulable as market power increases, while distance may be harder to control due to real estate effects, and ratings are almost fully out of control.

Question 2

Part 1

```
logit_data <- main_long %>%
  group_by(market) %>%
  mutate(
    s_out = 1 - sum(s),
```

```

    logit_s = log(s) - log(s_out)
  ) %>%
  ungroup()

mnl1 <- feols(
  logit_s ~ x + p,
  data = logit_data
)
summary(mnl1)

```

OLS estimation, Dep. Var.: logit_s

Observations: 284

Standard-errors: IID

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-4.744482	0.199805	-23.74556	< 2.2e-16 ***
x	0.059056	0.022765	2.59414	0.0099801 **
p	0.701882	0.037148	18.89406	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

RMSE: 0.546478 Adj. R2: 0.591078

Part 2

```

logit_data <- logit_data %>%
  group_by(market) %>%
  mutate(
    zp = mc
  ) %>%
  ungroup()

mnl2 <- ivreg(
  logit_s ~ x + p | x + zp,
  data = logit_data
)
summary(mnl2)

```

Call:

```
ivreg(formula = logit_s ~ x + p | x + zp, data = logit_data)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-3.11303	-0.87252	-0.03998	0.63885	4.13642

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.70956	2.22696	0.768	0.44333
x	0.24623	0.08168	3.015	0.00281 **
p	-0.56814	0.43724	-1.299	0.19488

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.248 on 281 degrees of freedom

Multiple R-Squared: -1.095, Adjusted R-squared: -1.11

Wald test: 6.085 on 2 and 281 DF, p-value: 0.002588

Part 3

```
logit_data <- logit_data %>%
  group_by(prod, market) %>%
  mutate(
    ad = ave_dist
  ) %>%
  ungroup()

mnl3 <- ivreg(
  logit_s ~ x + p + ad | x + ad + zp,
  data = logit_data
)
summary(mnl3)
```

Call:

ivreg(formula = logit_s ~ x + p + ad | x + ad + zp, data = logit_data)

Residuals:

	Min	1Q	Median	3Q	Max
	-3.11693	-0.86123	-0.07583	0.65975	4.13945

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.90340	2.24620	0.847	0.39750
x	0.24942	0.08221	3.034	0.00264 **
p	-0.58279	0.44010	-1.324	0.18651
ad	-0.14442	0.08312	-1.737	0.08341 .

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.255 on 280 degrees of freedom

Multiple R-Squared: -1.111, Adjusted R-squared: -1.134

Wald test: 4.968 on 3 and 280 DF, p-value: 0.002249

```
stargazer(mnl1, mnl2, mnl3,
  type = "text",
  dep.var.labels = "Logit Market Share",
  column.labels = c("No Instrument", "Instrument for Price",
    "Instrument for Price and Distance"),
  covariate.labels = c("Rating (x)", "Price (p)",
    "Average Distance (ad)"),
  stars = c(0.01, 0.05, 0.1),
  digits = 3,
  out = here("Y2S1/I0/PS3/Outputs/reg_results.txt"))
```

% Error: Unrecognized object type.

In the first model, we see a clear misspecification, since price's coefficient is positive. However, upon instrumenting for price, we flip the sign and get a negative coefficient. But price is not statistically significant, indicating a weak instrument. Finally, adding distance as an instrument, we see that both price and distance have negative signs, which is to be expected, but neither is statistically significant.

If the true values are the values set for simulation as laid out in part 1, price is biased negatively, distance is slightly biased negatively, and characteristics are slightly biased positively. Thus, all parameters estimated through multinomial logit are overstating the effect of these variables when in the case of complete information.

Question 3

```
search_dat <- read_csv(here("Y2S1/I0/PS3/Data/search_set_data.csv"))
```

Rows: 1136 Columns: 5

-- Column specification -----

Delimiter: ","

dbl (5): market, prod1, prod2, prod3, prod4

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
s_mat <- search_dat %>%  
  filter(market == min(market, na.rm = TRUE)) %>%  
  select(prod1, prod2, prod3, prod4) %>%  
  as.matrix()
```

```
stopifnot(nrow(s_mat) == 16, ncol(s_mat) == 4)
```

```
q4_df <- main_long %>%  
  mutate(prod = as.integer(prod)) %>%  
  arrange(market, prod)
```

```
q4_df <- q4_df %>%  
  group_by(market) %>%  
  filter(s > 0) %>%  
  ungroup()
```

```
n <- nrow(q4_df)  
Z <- model.matrix(~ x + mc + ave_dist, data = q4_df)  
W <- solve(crossprod(Z) / n)
```

```
eps <- 1e-12
```

```
delta_0 <- q4_df %>%  
  group_by(market) %>%  
  mutate(  
    s0 = 1 - sum(s),  
    delta_0 = log(s) - log(s0)  
  ) %>%
```

```

ungroup() %>%
pull(delta_0)

idx_vec <- split(seq_len(n), q4_df$market)

pi_func <- function(mu, ave_dist, eps = 1e-12) {
  pi <- plogis(10 + (mu * ave_dist))
  pmin(pmax(pi, eps), 1 - eps)
}

predict_share_func <- function(delta, pi, s_mat) {
  fixer <- max(0, max(delta))
  exp_fixed <- exp(delta - fixer)
  denom <- exp(-fixer) + as.vector(s_mat %*% exp_fixed)

  logP <- as.vector(s_mat %*% log(pi) + (1 - s_mat) %*% log(1 - pi))
  P <- exp(logP)

  con_P <- sweep(s_mat, 2, exp_fixed, `*`)
  con_P <- con_P / denom
  colSums(con_P * P)
}

con_map <- function(mu, delta_start, df, idx_vec, s_mat,
                    tol = 1e-10, max_iter = 1000) {

  eps <- 1e-12

  s_obs <- pmax(df$s, eps)
  pi_all <- pi_func(mu, df$ave_dist)

  delta <- delta_start

  for (i in seq_len(max_iter)) {
    s_hat <- numeric(length(delta))
    for (g in seq_along(idx_vec)) {
      idx <- idx_vec[[g]]
      s_hat[idx] <- predict_share_func(delta[idx], pi_all[idx], s_mat)
    }
    s_hat <- pmax(s_hat, eps)

    delta_new <- delta + log(s_obs) - log(pmax(s_hat, 1e-12))
  }
}

```

```

    diff <- max(abs(delta_new - delta))

    delta <- delta_new
    if (is.finite(diff) && diff < tol) {
      return(list(delta = delta, converged = TRUE, iter = i, diff = diff))
    }
  }
  list(delta = delta, converged = FALSE, iter = max_iter, diff = diff)
}
delta_last <- delta_0
gmm_exog <- function(theta) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  alpha <- theta[3]
  mu <- theta[4]

  cm <- con_map(
    mu,
    delta_last,
    q4_df,
    idx_vec,
    s_mat, tol = 1e-12, max_iter = 1000)

  if (!cm$converged || any(!is.finite(cm$delta)) || max(abs(cm$delta)) > 1e3) {
    return(1e12)
  }

  delta_last <- cm$delta

  xi <- cm$delta
  Q <- crossprod(Z, xi) / n
  obj <- as.numeric(t(Q) %*% W %*% Q)

  if (!is.finite(obj)) obj <- 1e12
  obj
}

if (exists("m3")) {
  b <- coef(m3)
  theta_start <- c(
    beta0 = unname(b["(Intercept)"]),
    beta1 = unname(b["x"]),

```

```

        alpha = unname(b["p"]),
        mu = -1.0)
} else {
  theta_start <- c(
    beta0 = 1.0,
    beta1 = 0.2,
    alpha = -0.25,
    mu = 0
  )
}

```

```

q4_df %>%
  group_by(market) %>%
  summarise(
    sum_s = sum(s),
    min_s = min(s),
    .groups = "drop"
  ) %>%
  arrange(desc(sum_s))

```

```

# A tibble: 71 x 3
  market sum_s min_s
  <dbl> <dbl> <dbl>
1      33 0.888 0.034
2      36 0.866 0.062
3       1 0.861 0.098
4      29 0.84  0.059
5      38 0.838 0.037
6      45 0.833 0.068
7      69 0.828 0.007
8       9 0.824 0.023
9      58 0.822 0.016
10     51 0.799 0.012
# i 61 more rows

```

```

set.seed(0219)
gmm_opt <- optim(
  par = theta_start,
  fn = gmm_exog,
  method = "Nelder-Mead",
  control = list(maxit = 5000, reltol = 1e-8)
)

```

```
)
```

```
theta_hat <- gmm_opt$par  
theta_hat
```

```
      beta0      beta1      alpha      mu  
1.5909270  0.5379885  0.5484737 -1.7378641
```

```
gmm_opt$value
```

```
[1] 0.5955458
```

```
gmm_opt$convergence
```

```
[1] 10
```

```
resolve <- con_map(  
  theta_hat["mu"],  
  delta_0,  
  q4_df,  
  idx_vec,  
  s_mat, tol = 1e-10, max_iter = 1000  
)
```

```
est_tab <- tibble(  
  term = c("Intercept", "Rating Coefficient", "Price Coefficient", "Mu"),  
  estimate = c(theta_hat["beta0"],  
               theta_hat["beta1"],  
               theta_hat["alpha"],  
               theta_hat["mu"]),  
  true = c(1, 0.2, -0.25, NA_real_)  
)
```

```
kable(est_tab, digits = 3, caption = "GMM Estimates vs. True Values")
```

Table 2: GMM Estimates vs. True Values

term	estimate	true
Intercept	1.591	1.00
Rating Coefficient	0.538	0.20
Price Coefficient	0.548	-0.25
Mu	-1.738	NA

Question 4

```
loc_dat <- read_csv(here("Y2S1/I0/PS3/Data/distance.csv"))
```

```
Rows: 71000 Columns: 5
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
dbl (5): market, dist1, dist2, dist3, dist4
```

i Use ``spec()`` to retrieve the full column specification for this data.

i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
q5_df <- main_long %>%
  arrange(market, prod)
```

```
S_temp <- search_dat %>%
  filter(market == min(market, na.rm = TRUE)) %>%
  select(prod1, prod2, prod3, prod4) %>%
  as.matrix()
```

```
S <- S_temp[rowSums(S_temp) > 0, , drop = FALSE]
stopifnot(ncol(S) == 4)
```

```
loc_list <- split(loc_dat, loc_dat$market)
loc_list <- lapply(loc_list, function(ave_dist) {
  as.matrix(ave_dist[, c("dist1", "dist2", "dist3", "dist4")])
})
```

```
n <- nrow(q5_df)
idx_vec <- split(seq_len(n), q5_df$market)
```

```

Z <- model.matrix(~ x + mc + ave_dist, data = q5_df)
W <- solve(crossprod(Z) / n)

delta_0 <- q5_df %>%
  group_by(market) %>%
  mutate(
    s0 = 1 - sum(s),
    delta_0 = log(s) - log(s0)
  ) %>%
  ungroup() %>%
  pull(delta_0)

predict_share_endog <- function(delta, gamma, D, S) {
  K <- nrow(S)
  N <- nrow(D)

  m <- max(c(0, delta))
  exp_fixed <- exp(delta - m)
  sum_exp <- as.vector(S %*% exp_fixed)
  denom <- exp(-m) + sum_exp

  U_til <- m + log(denom)
  P_til <- sweep(S, 2, exp_fixed, `*`)
  P_til <- P_til / denom

  loc_sum <- D %*% t(S)
  v <- matrix(U_til, nrow = N, ncol = K, byrow = TRUE) + gamma * loc_sum

  vm <- pmax(0, apply(v, 1, max))
  exp_v <- exp(v - vm)
  denom_search <- exp(-vm) + rowSums(exp_v)
  P_search <- exp_v / denom_search

  P_choice <- P_search %*% P_til
  colMeans(P_choice)
}

con_map_endog <- function(gamma, delta_start, df, loc_list, S,
                          idx_vec, tol = 1e-10, max_iter = 1000) {

  delta <- delta_start
  s_obs <- df$s

```



```

markets <- names(idx_vec)

for (i in seq_len(max_iter)) {
  s_hat <- numeric(length(delta))
  for (m in markets){
    idx <- idx_vec[[m]]
    Dm <- loc_list[[m]]
    s_hat[idx] <- predict_share_endog(delta[idx], gamma, Dm, S)
  }
  delta_new <- delta + log(s_obs) - log(pmax(s_hat, 1e-12))
  diff <- max(abs(delta_new - delta))
  delta <- delta_new
}
list(delta = delta, converged = FALSE, iter = max_iter, diff = diff)
}

delta_last <- delta_0

gmm_obj_endog <- function(theta) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  alpha <- theta[3]
  gamma <- theta[4]

  cm_endog <- con_map_endog(
    gamma      = gamma,
    delta_start = delta_last,
    df          = q5_df,
    loc_list    = loc_list,
    S           = S,
    idx_vec     = idx_vec,
    tol         = 1e-12,
    max_iter    = 1000
  )

  delta_last <- cm_endog$delta
  xi <- cm_endog$delta - (beta0 + beta1 * q5_df$x + alpha * q5_df$p)
  Q <- crossprod(Z, xi) / n
  as.numeric(t(Q) %*% W %*% Q)
}

theta_start_5 <- c(beta0 = 1, beta1 = 0.2, alpha = -0.25, gamma = -0.1)

```

```
if (exists("theta_hat4")) {
  theta_start_5[c("beta0", "beta1",
                  "alpha")] <- theta_hat4[c("beta0", "beta1", "alpha")]
}
```

```
set.seed(0219)
gmm_opt_endog <- optim(
  par = theta_start_5,
  fn = gmm_obj_endog,
  method = "Nelder-Mead",
  control = list(maxit = 5000, reltol = 1e-8)
)

theta_hat5 <- gmm_opt_endog$par
theta_hat5
```

beta0	beta1	alpha	gamma
2.6080609	0.2502021	-0.5873779	-0.2381165

```
gmm_opt_endog$value
```

```
[1] 4.463681e-09
```

```
gmm_opt_endog$convergence
```

```
[1] 0
```

```
resolve_endog <- con_map_endog(
  gamma      = theta_hat5["gamma"],
  delta_start = delta_0,
  df         = q5_df,
  loc_list   = loc_list,
  S          = S,
  idx_vec    = idx_vec,
  tol        = 1e-10,
  max_iter   = 1000
)

est_tab5 <- tibble(
```

```

term = c("Intercept", "Rating Coefficient", "Price Coefficient", "Gamma"),
estimate = c(theta_hat5["beta0"],
              theta_hat5["beta1"],
              theta_hat5["alpha"],
              theta_hat5["gamma"]),
true = c(1, 0.2, -0.25, -0.1)
)
kable(est_tab5, digits = 3,
      caption = "GMM Estimates with Endogenous Search vs. True Values")

```

Table 3: GMM Estimates with Endogenous Search vs. True Values

term	estimate	true
Intercept	2.608	1.00
Rating Coefficient	0.250	0.20
Price Coefficient	-0.587	-0.25
Gamma	-0.238	-0.10